Tema 5 — Memoria Transaccional por Software (STM)

Fuentes: teórica 06-stm, apunte de la práctica 5 ("Memoria Transaccional"), Guía 5. Para la sintaxis de Haskell ver `05-haskell-minimo.md`.

---

1. La idea

Con locks, componer operaciones es un problema. El ejemplo de la teórica: `transferir` con dos `synchronized` (extraer de una cuenta, depositar en la otra). Dos transferencias en sentido opuesto entre las mismas cuentas → **deadlock**. Y si no tomás los dos locks, alguien ve un estado intermedio en el que la plata no está en ninguna cuenta.

STM propone escribir:

```
atomic {
  cuenta.extraer(monto);
  depositar(monto);
}
```

y que el sistema garantice que el bloque se ejecuta **como si fuera de una sola vez**, sin que vos elijas ni ordenes ningún lock.

> **En criollo:** cada thread trabaja en un borrador. Al final intenta pasar el borrador en limpio. Si mientras escribía alguien cambió algo de lo que él había leído, tira el borrador y empieza de nuevo.

2. Cómo funciona por dentro (optimista)

1. La transacción corre **sin bloquear a nadie** y anota en un **log local**: qué TVars leyó (y qué valor vio) y qué quiere escribir.
2. Las escrituras **no tocan la memoria** mientras corre: nadie ve estados intermedios. Si relee algo que ella misma escribió, ve su propio valor del log.
3. Al final, **commit**: valida que todo lo que leyó siga igual. - Si sigue igual → todas sus escrituras impactan juntas. **Ese instante es el punto de linealización.** - Si algo cambió (conflicto) → descarta el log y **re-ejecuta inmediatamente** desde el principio.

Ejemplo: A y B hacen `incrementar` sobre un contador en 0. Los dos leen 0. A commitea (queda 1). B valida, ve que el contador ya no es 0 → re-ejecuta, lee 1, escribe 2. **No se pierde ningún incremento.**

Consecuencia directa: **una transacción puede correr varias veces**. Por eso adentro no puede haber efectos que no se puedan deshacer (imprimir, leer teclado, `threadDelay`).

3. Por qué Haskell

La teoría quiere `atomic :: IO a -> IO a`, pero así nada impide que alguien escriba una `IORef` por afuera de la transacción, o que imprima adentro. Haskell lo arregla con **tipos**: crea un contexto nuevo, `STM`, separado de `IO`.

	Qué es
`TVar a`	variable transaccional (memoria compartida)
`STM a`	una transacción: una **receta** que, si se ejecuta atómicamente, da un `a`
`IO a`	una acción con efectos (imprimir, crear threads, ...)

- Las operaciones sobre `TVar` devuelven `STM`, así que **solo se pueden usar dentro de una transacción**.
- `print` devuelve `IO`, así que **no compila** dentro de un `do` de STM. El tipo garantiza que no hay efectos irreversibles adentro.
- La única puerta de STM a IO es `atomically`.

(Antes de STM, la teórica muestra `IORef`: variables mutables en IO, **no thread-safe**: `readIORef` + `writeIORef` son dos pasos, se pierden incrementos igual que en Java.)

4. La API (todo de `Control.Concurrent.STM`)

Función	Tipo	Para qué
`newTVar`	`a -> STM (TVar a)`	crear (dentro de una transacción)
`newTVarIO`	`a -> IO (TVar a)`	crear desde IO (en `main`)
`readTVar`	`TVar a -> STM a`	leer
`writeTVar`	`TVar a -> a -> STM ()`	escribir
`modifyTVar'`	`TVar a -> (a -> a) -> STM ()`	leer-aplicar-escribir (`modifyTVar' c (+1)`)
`readTVarIO`	`TVar a -> IO a`	leer desde IO, **sin garantías** (el valor puede cambiar ya en la línea siguiente). Solo para mirar al final.
`atomically`	`STM a -> IO a`	**ejecutar** la transacción, atómicamente
`retry`	`STM a`	abortar y dormir hasta que cambie algo que leí
`check`	`Bool -> STM ()`	`check c` = si `c` es falso, `retry`
`orElse`	`STM a -> STM a -> STM a`	probar la primera; si hace `retry`, probar la segunda

Threads (de `Control.Concurrent` y `Control.Concurrent.Async`):

Función	Para qué
`forkIO :: IO () -> IO ThreadId`	lanzar un thread. **Si `main` termina, mueren todos.**
`async :: IO a -> IO (Async a)`	lanzar un thread y quedarte con un "comprobante"
`wait :: Async a -> IO a`	esperar a que termine (el `join` de Java)
`threadDelay :: Int -> IO ()`	dormir (microsegundos). **No sirve para sincronizar.**
`hSetBuffering stdout LineBuffering`	(de `System.IO`) que las impresiones de distintos threads no se mezclen caracter a caracter

5. Esperar una condición: `retry` / `check` ⭐

Es el reemplazo de `while (!cond) wait()` de los monitores.

```
decrementarSiPuede :: TVar Int -> STM ()
decrementarSiPuede v = do
  actual <- readTVar v
  check (actual > 0)          -- la condición para SEGUIR (al revés que el if con retry)
  writeTVar v (actual - 1)
```

- `retry` aborta y descarta todo lo que hizo la transacción (como un conflicto).
- **Pero no reintenta enseguida** (leería lo mismo y volvería a fallar): **duerme al thread**. El runtime mira en el log qué TVars se leyeron antes del `retry`, y lo despierta cuando **otro thread commitea un cambio en alguna de ellas**. **No hay busy waiting.**
- **No hay `signal`.** Nadie avisa a mano: el que modifica la TVar despierta solo a los que la leyeron. No hay lost wake-ups ni "signal a la condición equivocada".
- Cuando lo despiertan, re-ejecuta **todo** desde el principio, así que la condición se re-chequea sola (el `while` de Mesa viene gratis).

`orElse`: `a1 \`orElse` a2`. Si a1 hace retry, se descartan sus cambios y se prueba a2. Si las dos hacen retry`, el thread duerme hasta que cambie algo leído por cualquiera de las dos. Sirve para **preferencias** y **prioridades** ("intentá con esta cuenta y si no con la otra").

6. Composición ⭐

La gran ventaja sobre los locks: **las operaciones que devuelven `STM` se pueden combinar en una transacción más grande sin tocarlas.**

```
transferir :: TVar Int -> TVar Int -> Int -> STM ()
transferir de a n = do
  extraer de n        -- cada una es una STM ()
  depositar a n
-- y el que la usa: atomically (transferir c1 c2 50)
```

- Con locks, `move(lista1, lista2, x)` a partir de `remove` y `add` deja un instante donde `x` no está en ninguna, o te obliga a tomar locks internos de afuera (riesgo de deadlock). Con STM es un `do` con las dos.
- **Regla de diseño:** las operaciones de una estructura devuelven `STM`, no `IO`. **El que las usa decide dónde va el `atomically`** (o sea, qué es atómico junto).
- Si una parte hace `retry`, se descarta **la transacción entera**, incluidos los cambios de las partes anteriores.

7. Trampas (las que más importan)

1. **Avisar y esperar en la misma transacción = deadlock.** (Ejemplo de la carrera del apunte.) Si hacés `modifyTVar' listos (+1)` y después `check largaron` en el **mismo** `atomically`, el `retry` descarta el incremento: nadie ve nunca que estás listo. **Una transacción que espera no le puede avisar nada a nadie.** Avisar y esperar van en dos `atomically` separados. (El runtime detecta el caso y tira `thread blocked indefinitely in an STM transaction`.)

2. **Leer y escribir en transacciones separadas = race condition.** Si leés en un `atomically` y escribís en otro, entre los dos puede pasar cualquier cosa (es el `IORef` de nuevo). Lo que tiene que ser atómico va en **un** `atomically`.
3. **Nada de IO adentro.** Calculá dentro, devolvé el resultado con `return`, e imprimí afuera: `haskell puesto <- atomically (do llegados <- readTVar llegadas writeTVar llegadas (llegados ++ [i]) return (length llegados + 1)) putStrLn ("Llegué " ++ show puesto)`
4. `main` **no espera a nadie.** Si `main` termina, se mueren los threads. Usá `async` + `wait` (no `threadDelay`, y no "espero a que la lista tenga n elementos", que no garantiza que el thread terminó lo que hacía después).
5. `readTVarIO` **no da garantías**: solo para mirar el resultado final, cuando ya nadie escribe.

8. Propiedades (para justificar)

- **Safety:** cada transacción es atómica y aislada; el punto de linealización es el commit. No hay estados intermedios visibles.
- **Sin deadlock:** no hay locks. (Ojo: sí puede quedar todo el mundo dormido en `retry` si la lógica está mal, ver trampa 1.)
- **Sin garantía de no-inanición:** una transacción larga puede reintentar para siempre si otras le cambian todo el tiempo lo que lee. La lista con STM del apunte reintenta si alguien escribe cualquier `next` del tramo que recorrió (validación más conservadora que la optimista de la teórica de listas; hand-over-hand sí era libre de inanición).
- **Simplicidad:** el código es el secuencial + `readTVar` / `writeTVar` + `atomically`. Tan simple como granularidad gruesa, se comporta como sincronización optimista (pero la validación la hace el runtime).

9. Traducción desde lo que ya sabés

Monitores / semáforos	STM
variables protegidas por el lock	`TVar`s (agrupadas en un `data` con campos)
`synchronized` método	función que devuelve `STM`, ejecutada con `atomically`
`while (!cond) wait(c)`	`x <- readTVar ...; check cond`
`signal` / `signalAll`	**no existe**: escribir la TVar despierta a los que la leyeron
`acquire()`	`n <- readTVar s; check (n > 0); writeTVar s (n-1)`
tomar k permisos atómicamente (no se puede con `acquire(k)`)	**sí se puede**: k decrementos (o uno de k) en la misma transacción
"intentá A, y si no se puede, B"	`a \` orElse` b`
versión no bloqueante (`tryAcquire`)	devolver `Bool` / `Maybe` en vez de hacer `retry`, o `(algo >> return True) \` orElse` return False`
`while (true) { ... }` del thread	función recursiva sin caso base, o `forever`
`join`	`wait`

10. Receta para encarar un ejercicio de STM ⭐

1. **¿Qué se comparte?** Cada cosa compartida es una `TVar` (contador → `TVar Int`, buffer → `TVar (Maybe t)`, colección → `TVar [a]`, flag → `TVar Bool`). Si son varias, agrupalas en un `data`. Se crean

en `main` con `newTVarIO` y se pasan por parámetro (no hay globales).
2. **¿Qué operaciones hay?** Cada una es una función `... -> STM algo`. Adentro: leer, `check` la condición para poder seguir, escribir.
3. **¿Cuándo tiene que esperar alguien?** Esa es tu condición de `check` (o la rama del `case` que hace `retry`). No escribas ningún signal.
4. **¿Qué tiene que pasar todo junto?** Eso va en **un** `atomically`. Lo que es "aviso" y "espera" va en dos distintos (trampa 1).
5. **¿Hay preferencias o prioridades?** `orElse`.
6. **Threads:** cada participante es una función `IO ()` recursiva (o con `forever`) que hace `atomically (...)` y después la parte de IO (imprimir, `threadDelay` para simular que trabaja).
7. **Justificá:** safety (atomicidad de cada transacción), no deadlock (no hay locks; chequear que nadie quede en `retry` para siempre), y aclarar que STM **no** garantiza ausencia de inanición.

Formas de escribir "esperá hasta que...":

```
-- con check: la condición para SEGUIR
n <- readTVar v
check (n > 0)

-- con case + retry: cuando el valor tiene forma (Maybe, lista)
m <- readTVar buf
case m of
  Nothing -> retry
  Just x  -> do writeTVar buf Nothing; return x
```

Un thread que repite para siempre:

```
consumidor :: TVar Int -> Int -> IO ()
consumidor bolsa i = forever $ do
  atomically (tomarPar bolsa)        -- la parte que sincroniza
  putStrLn ("consumidor " ++ show i ++ " tomó un par")
  threadDelay 100000                  -- simular que hace algo
```

11. Mapa de la Guía 5 (idea de cada ejercicio, no la solución)

Ej	Qué es	Herramienta clave	Ojo con
1a	buffer de 1, no bloqueante	`case` sobre `Maybe`, devolver `Bool` / `Maybe` en vez de `retry`	—
1b	programa con put y get concurrentes	`async` + `wait`, `newTVarIO (Just 0)`	el buffer **arranca lleno**: pensá qué se imprime según quién corra primero
1c	buffer bloqueante	`retry` en el caso que no puede seguir	—
1d	`read` sin tomar	igual que `get` bloqueante pero sin escribir `Nothing`	—
2	semáforo	`TVar Int`; `acquire` = `check (n > 0)` + decrementar	tomar **dos** permisos atómicamente sí se puede: dos `acquire` en el mismo `atomically` (a diferencia de la cátedra con semáforos posta)
3	bolitas	invariante `bolitas <= generadores`: el generador hace `check (bolitas < generadores)`; el consumidor `check (bolitas >= 2)`	cantidad de jugadores no fija: los generadores **entran y salen** (contador en `TVar`), y al salir no pueden romper el invariante
4	fumadores	mesa como `TVar` con los ingredientes; cada fumador hace `check` de que están los dos que le faltan	el proveedor tiene que esperar a que la mesa se vacíe antes de poner más
5	pizzería	`tomarGrande \` orElse` tomarDosChicas`	las dos chicas se toman **en la misma** transacción (si hay una sola, no se lleva ninguna)
6	pila concurrente	`TVar [a]`; push = cons, pop = `retry` (o `Maybe`) si está vacía	—
7a	cola no acotada	`TVar [a]`; `desencolarSi` = versión con `Maybe`	encolar al final (`++ [x]`)
7b	mover entre colas	**composición**: `atomically (desencolar c1 >>= encolar c2)`	es el ejemplo de por qué las ops devuelven `STM` y no `IO`
7c	cola con prioridad	`orElse` encadenado: alta, si no media, si no baja	si las tres hacen `retry`, el thread duerme hasta que cambie cualquiera
8a	RWLock	lector: `check (not escritor)` + incrementar; escritor: `check (not escritor && lectores == 0)` + marcar	liberar es simplemente escribir; no hay signal
8b	RWLock con máximo de lectores	agregar el máximo al estado; `check (lectores < max)`	el máximo se fija al crear: va en el `data` (cambia `newRWLock`)
9	filósofos	tenedores como buffers/ `TVar Bool`; tomar **los dos** en la misma transacción	no hay deadlock (es atómico), pero mencioná que puede haber inanición

12. Patrones para la Guía 5 (resumen)

- **Buffer de 1 lugar** (Ej 1): `TVar (Maybe t)`. `Nothing` = vacío, `Just x` = lleno. Versión no bloqueante: devuelve `Bool` / `Maybe`. Versión bloqueante: `case` sobre el contenido + `retry` en el caso que no puede seguir.

- **Semáforo** (Ej 2): `TVar Int` + el `acquire` de la tabla. Dos permisos atómicos: dos `acquire` dentro del mismo `atomically` (o un `check (n >= 2)`).
- **Contador con tope / condición de invariante** (bolitas, Ej 3; RWLock, Ej 8): `check` sobre la condición del invariante antes de modificar.
- **Tomar varias cosas juntas** (fumadores Ej 4, dos chicas Ej 5, filósofos Ej 9): todo en una transacción; si falta algo, `retry` y no se toma nada.
- **Preferencias** (pizzería, Ej 5: grande o, si no, dos chicas): `orElse`.
- **Prioridades** (cola con prioridad, Ej 7c): `orElse` encadenado, de mayor a menor prioridad. Si todas hacen `retry`, el thread duerme.
- **Composición** (Ej 7b): combinar operaciones `STM` existentes en un `do` y poner un solo `atomically` afuera.
- **Esqueleto de un programa:** ```haskell import Control.Concurrent import Control.Concurrent.STM import Control.Concurrent.Async import System.IO

main :: IO () main = do hSetBuffering stdout LineBuffering estado <- newTVarIO valorInicial hilos <- mapM (\i -> async (trabajador estado i)) [1..n] mapM_ wait hilos `` - **Correrlo:**` runghc archivo.hs `(necesita` cabal install --lib stm async `una vez). Con ghci/runghc los threads se intercalan en un solo núcleo; para paralelismo real:` ghc -threaded archivo.hs `y` ./archivo +RTS -N4`.